VFR Assembler Specification v1.0

Assembly Language and Binary Format for the VFR Integer Processor

Registry: [?]

Series Path: [CKS-0-2026] → [CKS-MATH-0-2026] → [CKS-MATH-1-2026] → [CKS-MATH-10-2026] → [CKS-MATH-104-2026] → [?] → [?]

Parent Framework: [CKS-0-2026]

DOI: 10.5281/zenodo.zzz

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological "Truth" is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google's Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [CKS-LEX-12-2026]

---

1. Overview

The VFR Assembler (`vfra`) translates human-readable assembly source into binary batch streams executable by VFR cores. It is the first tool in the development chain — all ISA testing, FPGA validation, and early software development depend on it.

Source: .vfra (text) → Assembler → Output: .vfrb (binary batch stream)

The assembler is a simple two-pass tool. Pass one collects labels and computes addresses. Pass two encodes instructions and data. No optimization. No register allocation. No macro expansion. What you write is what executes.

Target implementation: approximately 500 lines of Zig. Runs on the development PC for cross-assembly, or on the ARM for on-device assembly.

---

2. Source File Format

2.1 File Extension

`.vfra` — VFR Assembly source.

2.2 Character Set

ASCII only. One statement per line. No line length limit. Lines terminated by newline (`') or carriage return + newline (`').

2.3 Comments

Semicolons to end of line:

ADD V1 V2 V3 ; this is a comment

; this entire line is a comment

2.4 Whitespace

Spaces and tabs separate tokens. Leading and trailing whitespace is ignored. Multiple spaces between tokens are equivalent to one.

2.5 Case Sensitivity

Instruction mnemonics are uppercase: `ADD`, `SUB`, `BLOAD`. Register names are uppercase: `V0`, `R15`. Directives are lowercase with leading dot: `.batch`, `.data`. Labels are case-sensitive and can be any case: `loop:`, `Done:`, `MAIN:`.

---

3. Literals

3.1 Decimal Integers

42

-17

0

1000000

Default representation. Signed. Range: -2,147,483,648 to 2,147,483,647 (i32).

3.2 Hexadecimal Integers

0x0000

0xFF

0x8000_0000

Prefix `0x`. Underscores permitted for readability. Always unsigned interpretation, cast to i32.

3.3 Binary Integers

0b00011111

0b1000_0000

Prefix `0b`. Underscores permitted.

3.4 Character Literals

'A' ; = 65

'Z' ; = 90

Single character in single quotes. Value is the Unicode codepoint as i32.

---

4. Registers

4.1 Value Registers

V0 V1 V2 V3 V4 V5 V6 V7

V8 V9 V10 V11 V12 V13 V14 V15

16 registers, each i32. Encoded as 4-bit index (0-15).

4.2 Remainder Registers

R0 R1 R2 R3 R4 R5 R6 R7

R8 R9 R10 R11 R12 R13 R14 R15

16 registers, each i8. Paired with V registers by index. R3 is the remainder partner of V3.

4.3 Batch Control Registers

Not directly addressable by name in most instructions. Loaded implicitly by `BLOAD`, read implicitly by `BNEXT` and `BADDR`. Visible in the debug output:

BF — batch factor (i16)

BC — batch count (u32)

BD — batch depth (u8)

BI — batch index (u32)

4.4 Memory Address Register

MA — memory address (u32)

Set implicitly by `BLOAD` (to the batch data start). Used implicitly by `BADDR`.

---

5. Labels

5.1 Definition

A label is a name followed by a colon, alone on its line or before an instruction:

loop:

ADD V1 V2 V3

done: HALT

5.2 Usage

Labels are used as operands in jump and branch instructions:

JMP loop

JEQ done

JDONE end_batch

5.3 Scope

Labels are global within a single source file. Duplicate label names are an assembler error.

5.4 Resolution

Labels resolve to byte offsets from the start of the current code batch. The assembler computes these in pass one and encodes them into the 26-bit address field of jump instructions in pass two.

---

6. Directives

Directives control the structure of the output binary. They do not produce instructions.

6.1 .batch — Begin a Batch

.batch F= count= depth=

Emits a 7-byte batch header to the output. All subsequent `.data` lines or instructions belong to this batch until the next `.batch` or `.end` directive.

.batch F=37 count=1000 depth=48 ; entity batch

.batch F=0 count=1 depth=1 ; code batch (F=0 convention for code)

.batch F=47 count=500 depth=2 ; fact batch for predicate 47

F=0 is the convention for code batches. The assembler does not enforce this — any F value is valid.

6.2 .data — Emit Raw VFR Pairs

.data [ ] …

Emits one or more VFR pairs as raw data (i32 + i8, 5 bytes each). Multiple pairs on one line are emitted sequentially.

.data 10 0 ; one VFR pair: V=10, R=0

.data 100 3 200 7 300 0 ; three VFR pairs

.data 0x1F 0 -50 -3 ; hex and negative values

6.3 .zero — Emit Zero-Filled VFR Pairs

.zero

Emits `count` VFR pairs of `[0, 0]`. Used to reserve space.

.zero 48 ; 48 zero VFR pairs (one empty entity)

.zero 1000 ; reserve 1000 pairs

6.4 .end — End of Stream

.end

Emits a 7-byte zero header `[0, 0, 0]` marking the end of the batch stream. Every `.vfra` file should end with `.end`.

6.5 .include — Include Another Source File

.include “math_lib.vfra”

Inserts the contents of the named file at this point. Paths are relative to the current file. Recursive includes are an error.

6.6 .const — Named Constant

.const ENTITY_DEPTH 48

.const HEALTH_OFFSET 30

.const MAX_ENTITIES 10000

Defines a name that substitutes its integer value wherever it appears. Constants are not stored in the output. They exist only during assembly.

.const HEALTH 30

LDVR V1 R1 [V0+HEALTH] ; assembles as [V0+30]

6.7 .origin — Set Address Counter

.origin 0x1000

Sets the current address counter to the specified value. Affects label resolution for subsequent code. Used when code or data must be placed at a specific BRAM address.

---

7. Instructions

7.1 Arithmetic

ADD Vd Va Vb ; Vd = Va + Vb

SUB Vd Va Vb ; Vd = Va - Vb

MUL Vd Va Vb ; Vd = Va × Vb

ADDR Rd Ra Rb ; Rd = Ra + Rb (i8 remainder add)

SUBR Rd Ra Rb ; Rd = Ra - Rb (i8 remainder sub)

7.2 Bitwise

AND Vd Va Vb ; Vd = Va & Vb

OR Vd Va Vb ; Vd = Va | Vb

XOR Vd Va Vb ; Vd = Va ^ Vb

NOT Vd Va ; Vd = ~Va

7.3 Shift

SHR Vd Va ; Vd = Va » imm

SHL Vd Va ; Vd = Va « imm

SHR5 Vd Va ; Vd = Va » 5 (base-32 divide)

SHL5 Vd Va ; Vd = Va « 5 (base-32 multiply)

Immediate is a literal integer (0-31 for shift amounts).

7.4 VFR Operations

DECOMP Vd Rd Va ; Vd = Va » 5, Rd = Va & 0x1F

RECOMP Vd Va Ra ; Vd = (Va « 5) | (Ra & 0x1F)

RNORM Rd Ra ; Rd = Ra mod 32 (normalized to -16..15)

RACCUM Rd Ra Rb ; Rd = Ra + Rb, set OV flag if overflow

SCALE Vd Va ; Vd = Va « (imm × 5)

7.5 Compare

CMP Va Vb ; set EQ/LT/GT flags from i32 compare

CMPR Ra Rb ; set EQ/LT/GT flags from i8 compare

7.6 Load / Store

LDV Vd [Va+] ; load i32 from local memory at Va + offset

LDR Rd [Va+] ; load i8 from local memory at Va + offset

STV [Va+] Vs ; store i32 to local memory at Va + offset

STR [Va+] Rs ; store i8 to local memory at Va + offset

LDVR Vd Rd [Va+] ; load VFR pair (5 bytes) from Va + offset

STVR [Va+] Vs Rs ; store VFR pair (5 bytes) to Va + offset

Offset is a literal integer in bytes. If omitted, offset is 0:

LDV V1 [V0] ; offset = 0

LDV V1 [V0+30] ; offset = 30 bytes

LDVR V1 R1 [V0+HEALTH] ; using named constant

Square brackets are required around the memory operand.

7.7 Batch Control

BLOAD [] ; load 7-byte header at address into BF,BC,BD; reset BI=0; set MA

BNEXT ; BI += 1; set DONE flag if BI == BC

BADDR Vd ; Vd = MA + 7 + (BI × BD × 5)

BLOAD address is a literal or label:

BLOAD [0x0000] ; load header at BRAM address 0

BLOAD [entity_data] ; load header at label address

7.8 Transfer (DMA)

TSEND [Va] ; send len bytes from local Va to target core

TRECV [Va] ; receive len bytes into local Va

TWAIT ; wait for pending TSEND to complete

TDONE ; signal transfer complete

Length and core ID are literal integers:

TSEND [V0] 240 5 ; send 240 bytes from address in V0 to core 5

TRECV [V1] 240 ; receive 240 bytes into address in V1

7.9 Control Flow

JMP ; unconditional jump

JEQ ; jump if EQ flag set

JLT ; jump if LT flag set

JGT ; jump if GT flag set

JDONE ; jump if DONE flag set (batch complete)

HALT ; stop core execution

---

8. Instruction Encoding

All instructions encode to 32 bits (4 bytes) per the ISA specification. The assembler produces these encodings:

8.1 Type A — Three Register

Used by: ADD, SUB, MUL, ADDR, SUBR, AND, OR, XOR

[opcode:6] [Vd:4] [Va:4] [Vb:4] [unused:14]

Bits 31-26: opcode

Bits 25-22: destination register

Bits 21-18: source A register

Bits 17-14: source B register

Bits 13-0: zero

8.2 Type B — Two Register + Immediate

Used by: SHR, SHL, SCALE

[opcode:6] [Vd:4] [Va:4] [imm:18]

Bits 31-26: opcode

Bits 25-22: destination register

Bits 21-18: source register

Bits 17-0: immediate value

8.3 Type C — VFR Pair

Used by: DECOMP, RECOMP, LDVR, STVR

[opcode:6] [Vd:4] [Rd:4] [Va:4] [unused:14]

Bits 31-26: opcode

Bits 25-22: V destination/source

Bits 21-18: R destination/source

Bits 17-14: V source/address base

Bits 13-0: zero (or offset for load/store)

8.4 Type D — Memory

Used by: LDV, LDR, STV, STR

[opcode:6] [reg:4] [offset:22]

Bits 31-26: opcode

Bits 25-22: register (source or destination)

Bits 21-0: byte offset (unsigned, 0 to 4,194,303)

For load/store with base register:

[opcode:6] [reg:4] [base:4] [offset:18]

Bits 31-26: opcode

Bits 25-22: destination/source register

Bits 21-18: base address register

Bits 17-0: byte offset

8.5 Type E — Transfer

Used by: TSEND, TRECV

[opcode:6] [core:10] [len:16]

Bits 31-26: opcode

Bits 25-16: core ID (0-1023)

Bits 15-0: transfer length in bytes (0-65535)

8.6 Type F — Jump

Used by: JMP, JEQ, JLT, JGT, JDONE

[opcode:6] [addr:26]

Bits 31-26: opcode

Bits 25-0: target address (instruction offset from batch start)

8.7 Type G — No Operands

Used by: HALT, BNEXT, TWAIT, TDONE

[opcode:6] [unused:26]

Bits 31-26: opcode

Bits 25-0: zero

---

9. Binary Output Format

9.1 File Extension

`.vfrb` — VFR Binary batch stream.

9.2 Structure

The output file is a sequence of batches followed by an end-of-stream marker:

[7-byte header] [data or instructions…]

[7-byte header] [data or instructions…]

…

[7 zero bytes] end of stream

9.3 Batch Header

Byte 0-1: F (i16, little-endian)

Byte 2-5: count (u32, little-endian)

Byte 6: depth (u8)

9.4 Data Batches

After a `.batch` directive followed by `.data` lines, the output contains:

[header: 7 bytes]

[VFR pair: 5 bytes] $\times$ (count $\times$ depth)

Each VFR pair:

Byte 0-3: V (i32, little-endian)

Byte 4: R (i8)

9.5 Code Batches

After a `.batch` directive followed by instructions, the output contains:

[header: 7 bytes]

[instruction: 4 bytes] $\times$ count

Code batches conventionally use F=0 and depth=1. Count is the number of instructions. Each instruction is 4 bytes. The assembler counts instructions between `.batch` and the next `.batch` or `.end` and writes the correct count into the header.

9.6 Byte Order

All multi-byte values are little-endian, matching the ARM Cortex-A9 native byte order and the FPGA's expected format.

---

10. Assembly Process

10.1 Pass One — Label Collection

The assembler reads the entire source file. For each line:

- If a `.batch` directive: record the start of a new batch, reset address counter within the batch.
- If a label definition: record the label name and current address.
- If an instruction: advance address counter by 4 bytes.
- If a `.data` directive: advance address counter by 5 bytes per VFR pair.

- If a `.zero` directive: advance address counter by 5 bytes × count.
- If a `.const` directive: record the name and value in the constant table.

At the end of pass one, all label addresses are known.

10.2 Pass Two — Encoding

The assembler reads the source file again. For each line:

- If a `.batch` directive: emit the 7-byte header. For code batches, the count field is filled in after counting instructions (back-patched or pre-counted from pass one).
- If an instruction: encode to 32 bits per the encoding rules. Resolve label references to addresses from pass one. Resolve named constants to their values.
- If a `.data` directive: emit VFR pairs as raw bytes.
- If a `.zero` directive: emit zero bytes.
- If a `.end` directive: emit 7 zero bytes.

10.3 Error Handling

The assembler reports errors with file name, line number, and description:

Error: test.vfra:15: undefined label 'looop' (did you mean 'loop'?)

Error: test.vfra:23: register V16 does not exist (valid: V0-V15)

Error: test.vfra:31: immediate value 50 exceeds shift range (max 31)

Error: test.vfra:8: duplicate label 'done' (first defined at line 3)

Error: test.vfra:42: .data in code batch (use .batch for data)

Errors halt assembly. No partial output is produced. Fix all errors, assemble again.

10.4 Warnings

Warning: test.vfra:20: label 'unused_label' defined but never referenced

Warning: test.vfra:35: .batch count=100 but only 95 data entries provided

Warnings do not halt assembly. Output is still produced.

---

11. Complete Example Programs

11.1 Minimal Test — Add Two Constants

; minimal.vfra

; Adds two constants, stores result, halts.

```
.const RESULT_ADDR 0x100
.batch F=0 count=1 depth=1
LDV V1 [0x00] ; load first value (placed at 0x00 by host)
LDV V2 [0x04] ; load second value
ADD V3 V1 V2 ; add
STV [RESULT_ADDR] V3 ; store result
HALT
.end
```

Binary output: 7-byte header (F=0, count=5, depth=1) + 20 bytes of instructions + 7-byte end marker = 34 bytes total.

11.2 Batch Processing — Element-Wise Add

```
; batch_add.vfra
; Adds corresponding values from two fields in each entity.
; Entity depth=3: [field_a, field_b, result]
.batch F=1 count=4 depth=3
.data 10 0 20 0 0 0 ; entity 0
.data 30 0 40 0 0 0 ; entity 1
.data 50 0 60 0 0 0 ; entity 2
.data 70 0 80 0 0 0 ; entity 3
.batch F=0 count=1 depth=1
BLOAD [0x0000] ; load entity batch header
loop:
JDONE done
BADDR V0 ; V0 = address of current entity
LDVR V1 R1 [V0+0] ; load field_a
LDVR V2 R2 [V0+5] ; load field_b (offset 5 = one VFR pair)
ADD V3 V1 V2 ; add values
ADDR R3 R1 R2 ; add remainders
STVR [V0+10] V3 R3 ; store to result (offset 10 = two VFR pairs)
BNEXT
```

```
JMP loop
done:
HALT
.end
```

11.3 Conditional — Threshold Check

```
; threshold.vfra
; For each entity, if health < 50, set alert flag to 1.
.const HEALTH_OFF 30 ; pair index 6 × 5 bytes
.const ALERT_OFF 35 ; pair index 7 × 5 bytes
.const THRESHOLD 50
.batch F=37 count=100 depth=48
.zero 4800 ; 100 entities × 48 pairs (host fills real data)
.batch F=0 count=1 depth=1
BLOAD [0x0000]
loop:
JDONE done
BADDR V0
LDV V1 [V0+HEALTH_OFF] ; load health
LDV V2 [THRESHOLD] ; load threshold…
; Actually, we need immediate loads. Use a data region:
; For now, store threshold in a known location
; Better approach: load from a constants region
CMP V1 V2
JLT set_alert ; health < threshold
JMP next
set_alert:
LDV V3 [one_const] ; load the value 1
STV [V0+ALERT_OFF] V3 ; set alert = 1
next:
BNEXT
```

```
JMP loop
done:
HALT
; Constants stored in data region after code
.origin 0x2000
one_const:
.data 1 0
threshold_const:
.data 50 0
.end
```

11.4 VFR Decomposition

```
; decompose.vfra
; Decompose every value in a batch into quotient and remainder.
.batch F=32 count=8 depth=1
.data 0 0
.data 31 0
.data 32 0
.data 33 0
.data 63 0
.data 64 0
.data 100 0
.data 255 0
.batch F=0 count=1 depth=1
BLOAD [0x0000]
loop:
JDONE done
BADDR V0
LDV V1 [V0+0] ; load value
DECOMP V2 R2 V1 ; V2 = V1 » 5, R2 = V1 & 0x1F
STVR [V0+0] V2 R2 ; overwrite with decomposed form
```

```
BNEXT
JMP loop
done:
HALT
.end
; Expected results:
; 0 → V=0, R=0
; 31 → V=0, R=31
; 32 → V=1, R=0
; 33 → V=1, R=1
; 63 → V=1, R=31
; 64 → V=2, R=0
; 100 → V=3, R=4
; 255 → V=7, R=31
```

11.5 Multi-Batch Stream

```
; multi_batch.vfra
; Process two batches with different F values in sequence.
; First batch: positions at F=37
.batch F=37 count=3 depth=2
.data 100 0 200 0 ; entity 0: x=100, y=200
.data 150 0 250 0 ; entity 1
.data 300 0 400 0 ; entity 2
; Second batch: velocities at F=12
.batch F=12 count=3 depth=2
.data 5 0 -3 0 ; entity 0: vx=5, vy=-3
.data 0 0 10 0 ; entity 1
.data -2 0 7 0 ; entity 2
; Code: add velocity to position for each entity
.batch F=0 count=1 depth=1
; Load position batch
```

```
BLOAD [0x0000]
; Process positions
pos_loop:
JDONE pos_done
BADDR V0
LDVR V1 R1 [V0+0] ; load pos_x
LDVR V2 R2 [V0+5] ; load pos_y
; Velocity is at a known offset after position batch
; (Host pre-computes and provides velocity base address)
; For this example, assume V10 holds velocity batch base
; … simplified: real implementation uses BADDR on second batch
BNEXT
JMP pos_loop
pos_done:
HALT
.end
```

11.6 Prolog Fact Filtering

```
; fact_filter.vfra
; Filter fact batch for predicate 47 (has_target) where arg0 == entity 42.
.const ENTITY_42 42
; Fact batch: has_target(entity_id, target_id)
.batch F=47 count=5 depth=2
.data 42 0 17 0 ; entity 42 targets 17
.data 43 0 17 0 ; entity 43 targets 17
.data 42 0 22 0 ; entity 42 targets 22
.data 44 0 31 0 ; entity 44 targets 31
.data 42 0 8 0 ; entity 42 targets 8
; Results batch: matching target IDs
.batch F=100 count=0 depth=1 ; count filled at runtime
.zero 10 ; reserve space for up to 10 results
```

```
; Code
.batch F=0 count=1 depth=1
; Load source entity ID
LDV V10 [entity_id_addr] ; V10 = 42
; Load fact batch
BLOAD [0x0000]
LDV V11 [result_base] ; V11 = result write address
LDV V12 [zero_const] ; V12 = result count = 0
filter_loop:
JDONE filter_done
BADDR V0
LDV V1 [V0+0] ; load entity_id from fact
CMP V1 V10 ; compare with target entity
JEQ match
BNEXT
JMP filter_loop
match:
LDV V2 [V0+5] ; load target_id from matching fact
STV [V11+0] V2 ; write to result batch
ADD V11 V11 V3 ; advance result pointer (V3 = 5, stride)
ADD V12 V12 V4 ; increment result count (V4 = 1)
BNEXT
JMP filter_loop
filter_done:
; V12 holds count of matches
; Result batch at result_base contains matched target IDs
HALT
.origin 0x2000
entity_id_addr:
.data 42 0
result_base:
```

```
.data 0x1000 0 ; address of result batch data region
zero_const:
.data 0 0
.end
```

---

12. Assembler Command Line Interface

12.1 Usage

```
vfra <input.vfra> [-o <output.vfrb>] [-v] [-d]
```

12.2 Options

```
<input.vfra> Source file (required)
-o <output.vfrb> Output file (default: input name with .vfrb extension)
-v Verbose: print label addresses and batch sizes
-d Dump: print hex dump of output binary
```

12.3 Example Invocations

```
vfra test.vfra ; produces test.vfrb
vfra test.vfra -o boot.vfrb ; custom output name
vfra test.vfra -v ; verbose label listing
vfra test.vfra -d ; hex dump of output
```

12.4 Verbose Output

```
$ vfra batch_add.vfra -v
VFR Assembler v1.0
Source: batch_add.vfra
Pass 1: Label collection
loop: offset 0x04 (batch 1, instruction 1)
done: offset 0x28 (batch 1, instruction 10)
Batches:
Batch 0: F=1 count=4 depth=3 data 80 bytes
Batch 1: F=0 count=11 depth=1 code 44 bytes
```

Labels:

loop = 0x04

done = 0x28

Pass 2: Encoding

Total output: 138 bytes (80 data + 44 code + 14 headers + 7 end)

Output: batch_add.vfrb

12.5 Hex Dump Output

```
$ vfra minimal.vfra -d
0000: 00 00 05 00 00 00 01 ; header: F=0, count=5, depth=1
0007: [30 58 00 00] ; LDV V1 [0x00]
000B: [30 98 04 00] ; LDV V2 [0x04]
000F: [08 64 80 00] ; ADD V3 V1 V2
0013: [32 C1 00 00] ; STV [0x100] V3
0017: [00 00 00 00] ; HALT
001B: 00 00 00 00 00 00 00 ; end of stream
```

---

13. Integration with FPGA Testing

13.1 Test Workflow

1. Write test program in .vfra
2. Assemble: vfra test.vfra -o test.vfrb
3. Copy test.vfrb to SD card
4. ARM boot loader reads test.vfrb from SD card
5. ARM writes binary to DDR3 staging area
6. ARM triggers FPGA: ENTITY_ADDR = staging address, CONTROL = 1
7. FPGA batch dispatcher loads data to core BRAMs
8. Core(s) execute instructions
9. Core(s) signal done
10. ARM reads results from DDR3
11. ARM prints results over UART serial console

13.2 ISA Validation Suite

A set of `.vfra` test programs, one per opcode group:

tests/

├── test_arithmetic.vfra ADD, SUB, MUL, ADDR, SUBR with known results

├── test_bitwise.vfra AND, OR, XOR, NOT

├── test_shift.vfra SHR, SHL, SHR5, SHL5

├── test_vfr.vfra DECOMP, RECOMP, RNORM, RACCUM, SCALE

├── test_compare.vfra CMP, CMPR with all flag combinations

├── test_loadstore.vfra LDV, LDR, STV, STR, LDVR, STVR

├── test_batch.vfra BLOAD, BNEXT, BADDR loop

├── test_jump.vfra JMP, JEQ, JLT, JGT, JDONE

├── test_halt.vfra HALT stops core

├── test_edge_cases.vfra Zero, negative, overflow, max values

└── test_integration.vfra Complete batch processing program

Each test writes expected results to known BRAM addresses. The ARM reads those addresses and compares against expected values. Pass or fail reported over UART.

13.3 ARM Test Harness (Zig)

zig

fn runTest(test_name: []const u8, binary: []const u8, expected: []const u32) bool {

// Write binary to DDR3

writeToStaging(binary);

// Dispatch to FPGA

Reg.ENTITY_ADDR.* = STAGING_ADDR;

Reg.CONTROL.* = 1;

// Wait for completion

while (Reg.STATUS.* & 0x02 == 0) {}

// Read results

var pass = true;

for (expected, 0..) |exp, i| {

```
const actual = readFromDDR3(RESULT_ADDR + i * 4);

if (actual != exp) {

   uart.print("{s}: FAIL at index {d}: expected 0x{x}, got 0x{x}",

         .{test_name, i, exp, actual});

    pass = false;

}

}
```

if (pass) uart.print("{s}: PASS", .{test_name});

return pass;

}

---

14. Assembler Implementation Notes

14.1 Implementation Language

Zig. The assembler is a Zig program that runs on the development PC (cross-assembling) or on the ARM (native assembling).

14.2 Estimated Size

Tokenizer: ~80 lines (split lines into tokens)

Parser: ~150 lines (recognize directives and instructions)

Label collector: ~50 lines (pass one)

Encoder: ~200 lines (instruction encoding, pass two)

Binary writer: ~40 lines (output .vfrb format)

Main + CLI: ~30 lines

Error reporting: ~50 lines

Total: ~600 lines of Zig

14.3 No External Dependencies

The assembler uses only Zig standard library for file I/O and string processing. No parser generators. No regex. No external tools. Single file compilation: `zig build-exe`

`vfra.zig`.

---

15. Summary

Property	Specification
Source format	`.vfra` text, ASCII, one statement per line
Output format	`.vfrb` binary batch stream
Instructions	All 35 ISA instructions supported
Encoding	32-bit fixed-width per ISA spec
Directives	.batch, .data, .zero, .end, .include, .const, .origin
Labels	Global, resolve to byte offsets, used in jumps
Constants	Named integer substitution, assembly-time only
Literals	Decimal, hexadecimal (0x), binary (0b), character ('x')
Registers	V0-V15 (i32), R0-R15 (i8)
Passes	Two: label collection, then encoding
Error handling	Line-numbered errors, halt on error, warnings continue
Output byte order	Little-endian
Implementation	~600 lines of Zig
Dependencies	None (Zig standard library only)

VFR Assembler Specification v1.0

Companion document to ISA v1.0, Compiler v1.0, and FPGA Implementation v1.0

[**CKS-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[**CKS-MATH-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/C MATH-0-2026

[**CKS-MATH-1-2026**] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026

[**CKS-MATH-10-2026**] Grand Unification. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026

[**CKS-MATH-104-2026**] Grand Unification v23. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS
MATH-104-2026

[**CKS-LEX-12-2026**] Measurement Systems in the $\mathbb{Q}$-Substrate. Github: https://github.com/ghowland/cks/tree/main/paper
LEX-12-2026